

SteganoGAN Re-implementation: <https://github.com/shriyasudhakar/CS4782-Final-Project>
Shriya Sudhakar (ss3576), Nidhi Soma (ns848), Kiran Mitra (km936), Niti Goyal (ng459)

Introduction

Our project is a reimplementation of SteganoGAN: High Capacity Image Steganography with GANs by Zhang et. al, 2019. The main motivation behind SteganoGAN is to create a SoTA deep learning technique that stores more bits per pixel (larger messages) compared to traditional steganography methods and generalizes to more image sizes and message types compared to other deep-learning based methods. This paper implements a GAN-based method (encoder, decoder, critic, all using convolutional methods) to embed messages in images in an undetectable manner.

Chosen Result

We reproduced Table 1 from the paper, which compares the three encoder architectures (Basic, Residual, Dense) across data depths $D \in \{1, 3, 6\}$ using decoder accuracy, Reed Solomon Bits Per Pixel (RS-BPP), Peak Signal to Noise Ratio (PSNR), and Structural Similarity Index (SSIM). The data depth D controls how many bits of hidden message are packed into each pixel of the cover image. This result (**see Appendix A for Table 1**) demonstrates the paper’s key contribution of developing a deep-learning method to both preserve image realness during encoding and then decode the message accurately while being able to encode greater message payloads than achieved before. This table also the paper’s key claim that the dense encoder improves performance, serving as a foundation for our follow-up extensions. We also ran StegExpose (a traditional steganalysis method) on a subset of our models (which the paper’s authors also ran on their models).

Extensions: We ran additional experiments to test for paper’s generalizability and explore improvements: (1) We introduced a novel self-attention encoder and decoder. (2) We generated blurred and grayscale versions of our DIV2K validation set to check model performance. (3) We experimented with scaling MSE loss by 100 as we discovered authors did in their code. (4) We experimented with taking out critic and adversarial components entirely.

Methodology

Dataset: We replicated Table 1 using the DIV2K image dataset as our training set, a high-res (HR) dataset featuring diverse open scenery (due to COCO being large, and more cluttered images). We downloaded pre-split data from DIV2K’s HR train (800 images) and validation (100 images) sets.

Architecture: Encoders: Each model takes in a cover image of $H \times W$ and message of $D \times H \times W$. A convolution block (Conv) consists of: (1) Convolution layer with kernel size 3, stride 1, padding “same”, (2) Leaky ReLU activation (3) 2D BatchNorm. All encoders use a Conv block, then concatenate the message, then use another Conv block. For the Basic variant, we add two sequential Conv blocks and the output is the steganographic image. For the Residual variant, we also add the cover image to the last Conv block’s output. For the Dense variant, we add the intermediate outputs between each Conv block.

Decoder: Uses stacked convolutional layers, then outputs a tensor with the same message depth as the embedded message. **Critic:** CNN-based discriminator that outputs a realism score. **Training:** With 32 joint epochs, the critic learns to assign higher scores to real images and lower scores to steganographic images, while the encoder learns to make steganographic images that fool the critic. This encourages the encoded images to look natural and reduces visible artifacts. **Attention Encoder/Decoder:** Our novel Attention encoder and decoder extends the Dense encoder. Since convolution-based methods tend to have limited local receptive fields, we hypothesize that attention allows for globally-informed visualization of where to embed messages. After the third convolutional layer, we apply Window Multi-Head Self-Attention (similar to the Swin Transformer). The feature map is partitioned into 8×8 windows, and we compute Q, K, and V, using 1×1 convolutions, and scaled dot product attention across 64 positions and 4 heads. The output of attention then passes through BatchNorm, another Conv layer, and finally is added residually to the cover image. The decoder mirrors the encoder, so that it learns to look for messages in the same spatial regions.

Metrics: We measured the quality of the different trained variants of our steganography tool using Decoder Accuracy, RS-BPP, SSIM, and PSNR. The decoder accuracy tells us what fraction of hidden bits were recovered correctly. However, even if there is a low decoder accuracy, we typically use Reed-Solomon error correction to add redundancy and allow some bits to be flipped without corrupting

the message. RS-BPP measures how many bits we can actually interpret per pixel by accounting for this correction schema. PSNR measures how visually different the original image and steganographic image are, using the average squared pixel difference between the images. SSIM follows the same idea, but accounts for local patterns of brightness, contrast, and structure between the two images rather than individual raw pixels.

Results & Analysis (See Appendix B.1 for our reproduction of Table 1.)

Overall, our novel Attention architecture performed the best, and we also were able to reproduce similar trends to the paper’s results, specifically their best result, Dense at depth 6. Early on, we noticed consistently higher RS-BPP & Accuracy and lower PSNR, SSIM, which means that we are able to decode better, but our images look tampered. To account for this, we experimented with scaling the MSE loss between the real image and the steganographic image by 100 (as reported in their codebase, not paper). This greatly improved our image realness metrics, matching the paper’s reported results. **See Appendix B.1 for metrics.** Additionally, we removed the critic to understand how much it affects the training (after scaling the MSE loss). After re-training, we achieved very similar metrics without the critic at depths 1 and 6, showing that GAN-style training may not be absolutely necessary since the MSE loss with scaling dominates the loss function. We also noticed a tradeoff during training at the pareto-optimal frontier between bit encoding-decoding accuracy and similarity of the cover and steganographic images: as image realness (measured by PSNR & SSIM) decreases, the decoding accuracy and RS-BPP increases (**Appendix B.6**). This makes sense since it is easier to find relevant message bits in images that visually seem to be tampered/alterd. This demonstrates the fundamental problem in steganography as a whole: the balance between image realness and ability to locate and decode the message.

We also evaluated using StegExpose, an open-source traditional steganalysis tool that distinguishes a steganographic image from a cover image. The paper reports an auROC of 0.59 for the Dense encoder at D=6 (Figure 5), barely above random chance, meaning the model successfully fools StegExpose on in-distribution images. **See Appendix B.2 for image for StegExpose Figure from paper.** We reproduced this experiment with the Dense decoder at D=1,3,6 (without loss scaling) with one key change: we used 100 cover images from the COCO 2017 validation dataset even though our model was trained on DIV2K. Our auROC of 0.76 vs. the paper’s 0.59 means that our model is less effective at evading detection under a distribution shift. **See Appendix B.3 for our StegExpose figure.** This suggests that the architecture is not robust to such a distribution shift and it would be necessary to train SteganoGAN on a large variety of images to achieve the same performance on arbitrary real-world situations as the paper claims.

For our blurred & grayscale images experiment, we visualized the residual message by using colored spots to indicate which channel(s) the message is stored in and where the message is encoded. **See Appendix B.4 for residual images.** We observed that the grayscale residual encoded the message evenly across colored channels and focused on edges, since we see a lot of white silhouettes. The blurred residual focused more on color channels and avoided edges. This makes sense because it is noticeable if a grayscale image had more color in one channel, while it would be easy to notice a sharpened edge for a blurry image. This also showed that our model trained only on natural images was able to adapt to a similar modified distribution. **See Appendix B.5 for metrics.**

Reflections

Through this re-implementation, we learned that SteganoGAN is highly sensitive to the balance between message recovery, image similarity, and adversarial realism. While the core architecture is conceptually sound, we notice that small changes to the loss weights, data depth, and architecture choice can noticeably affect the tradeoff between decoding accuracy and image quality. A key takeaway from our experiments is that successful steganography is not just about maximizing decoder accuracy. The model also needs to preserve image quality (PSNR & SSIM), while maintaining a useful recovered payload rate. With more time and compute, we would test whether humans or language models can still recover the intended natural language message (not random bit-messages) even when the bit-level recovery is only partial. This would be useful because a message may remain understandable despite some corrupted bits.

References

- [1] Zhang, Kevin Alex, et al. "SteganoGAN: High Capacity Image Steganography with GANs." arXiv preprint arXiv:1901.03892 (2019).
- [2] Boehm, Benedikt. "Stegexpose - A Tool for Detecting LSB Steganography." arXiv preprint arXiv:1410.6656 (2014).
- [3] Liu, Ze, et al. "Swin Transformer: Hierarchical Vision Transformer using Shifted Windows." arXiv preprint arXiv:2103.14030 (2021).
- [4] Ignatov, Andrey and Timofte, Radu, et al. "PIRM challenge on perceptual image enhancement on smartphones: report". In *European Conference on Computer Vision (ECCV) Workshops* (2019).
- [5] Lin, T., et al. "Microsoft COCO: common objects in context." arXiv preprint arXiv:1405.0312 (2014).

Appendix A:

SteganoGAN													
Dataset	D	Accuracy			RS-BPP			PSNR			SSIM		
		Basic	Resid.	Dense	Basic	Resid	Dense	Basic	Resid.	Dense	Basic	Resid.	Dense
Div2K	1	0.95	0.99	1.00	0.91	0.99	0.99	24.52	41.68	41.60	0.70	0.96	0.95
	2	0.91	0.98	0.99	1.65	1.92	1.96	24.62	38.25	39.62	0.67	0.90	0.92
	3	0.82	0.92	0.94	1.92	2.52	2.63	25.03	36.67	36.52	0.69	0.85	0.85
	4	0.75	0.82	0.82	1.98	2.52	2.53	24.45	37.86	37.49	0.69	0.88	0.88
	5	0.69	0.74	0.75	1.86	2.39	2.50	24.90	39.45	38.65	0.70	0.90	0.90
	6	0.67	0.69	0.70	2.04	2.32	2.44	24.72	39.53	38.94	0.70	0.91	0.90

Appendix B.1

Depth	Accuracy				RS-BPP			
	Basic	Residual	Dense	Attention Enc + Dec	Basic	Residual	Dense	Attention Enc + Dec
1	0.9613	0.9473	0.9604	0.9955	0.942	0.8946	0.9208	0.9911
1 (scaled MSE by 100)	0.9465	0.9887	0.9806	0.993	0.893	0.9773	0.9612	0.9859
1 (MSE scaling + no critic loss)	0.9458	0.989	0.986	0.9972	0.8916	0.978	0.9721	0.9944
3	0.8603	0.9325	0.7969	0.99	1.75626	2.5952	1.7814	2.9401
3 (scaled MSE by 100)	0.71978	0.8643	0.8871	0.9171	1.3187	2.1857	2.3224	2.5029
6	0.71567	0.6221	0.7338	0.7975	2.5881	1.4647	2.806	3.5704
6 (scaled MSE by 100)	0.636	0.6748	0.6694	0.6826	1.6323	2.0973	2.0323	2.1916
6 (MSE scaling + no critic loss)	0.6392	0.679	0.6827	0.6971	1.6702	2.1476	2.1918	2.3656

Depth	PSNR				SSIM			
	Basic	Residual	Dense	Attention Enc + Dec	Basic	Residual	Dense	Attention Enc + Dec
1	24.692	31.67	28.86	33.63	0.697	0.8981	0.8276	0.9359
1 (scaled MSE by 100)	29.74	41.37	38.77	41.99	0.8506	0.98	0.9651	0.9856

1 (MSE scaling + no critic loss)	29.87	40.13	38.3	41.7	0.8692	0.9705	0.9584	0.9816
3	22.45348	23.64	24.68	25.38	0.52964	0.4965	0.5666	0.702
3 (scaled MSE by 100)	30.17127	37.43	35.27	37.37	0.88326	0.945	0.9279	0.9524
6	21.786	22.95	23.07	23.48	0.4761	0.4315	0.4402	0.4628
6 (scaled MSE by 100)	29.21	41.92	38.84	40.25	0.8696	0.98	0.9653	0.9751
6 (MSE scaling + no critic loss)	30.69	40.55	37.5	38.88	0.8952	0.9705	0.9553	0.9619

Appendix B.2

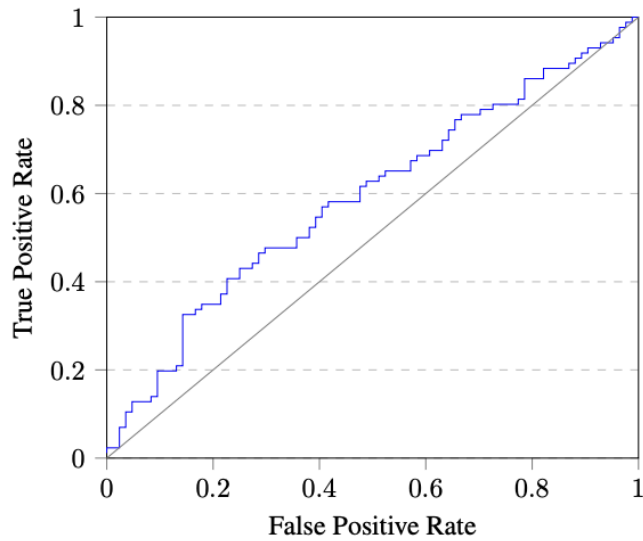
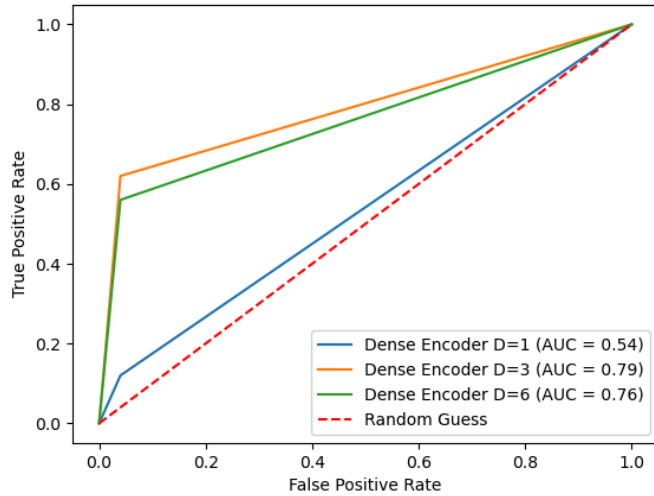


Figure 5. The receiver operating characteristic (ROC) curve produced by the StegExpose library for a set of 1000 steganographic images generated using the Dense architecture with a data depth of 6. The StegExpose library includes multiple steganalysis tools including *SamplePairs* (Dumitrescu et al., 2003), *RSAnalysis* (Fridrich et al., 2001), *ChiSquaredAttack* (Westfeld & Pfitzmann, 2000), and *PrimarySets* (Dumitrescu et al., 2002). The tool achieves an auROC of 0.59.

Appendix B.3

COCO Test Set ROC Curve with StegExpose Fusion (mean), threshold=0.2



Appendix B.4

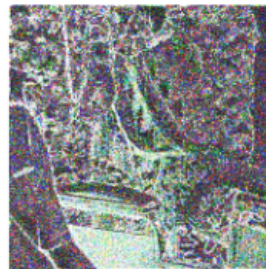
Original
image



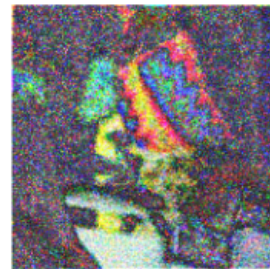
Original
residual



Grayscale
residual



Blurred
residual



Appendix B.5

Basic Encoder Table

	Accuracy	RS-BPP	PSNR	SSIM
Original, depth 6	0.71567	2.5881	21.78629	0.4761
Grayscale, depth 6	0.7201	2.6415	22.4235	0.4859
Blurred, depth 6	0.7258	2.7101	25.3727	0.5251

Appendix B.6 Metric trade-off

